

Trabajo Práctico 2 — Catan

[7507/9502] Algoritmos y Programación III
Curso Suarez
Segundo cuatrimestre de 2025

Alumno:	Padrón	Email
BASSANI, Santiago	113868	santiagobassani2000@gmail.com
BASUALDO, Pedro Juan	109792	jpbasualdo@fi.uba.ar
GUO, Fernando	101923	fguo@fi.uba.ar
LÓPEZ, Ignacio Daniel	103433	ilopez@fi.uba.ar

Índice

1. Introducción	2
2. Diagramas de clase	2
3. Detalles de implementación	4
3.1. Juego	4
3.2. Turnos	4
3.3. Tablero	4
3.4. Jugador	5
3.5. Banca	5
3.6. Dados	6
3.7. Terreno	6
3.8. Vertice	6
3.9. Arista	7
3.10. Ladron	7
3.11. Puerto, PuertoEspecifico2_1 y PuertoGenerico3_1	8
3.12. Puntaje y Bonificaciones	8
3.13. Cartas de Desarrollo	8
4. Justificación de los patrones de diseño aplicados	9
4.1. Patrón Singleton	9
4.2. Patrón Strategy	9
4.3. Patrón Template Method	9
4.4. Patrón Facade	9
4.5. Patrón Factory Method	10
5. Excepciones	10
6. Diagramas de secuencia	11

1. Introducción

El presente informe reúne la documentación de la solución para el segundo trabajo práctico de la materia Algoritmos y Programación III, el cual consiste en desarrollar el juego Catán en Java utilizando los conceptos del paradigma de la orientación a objetos vistos en el curso.

2. Diagramas de clase

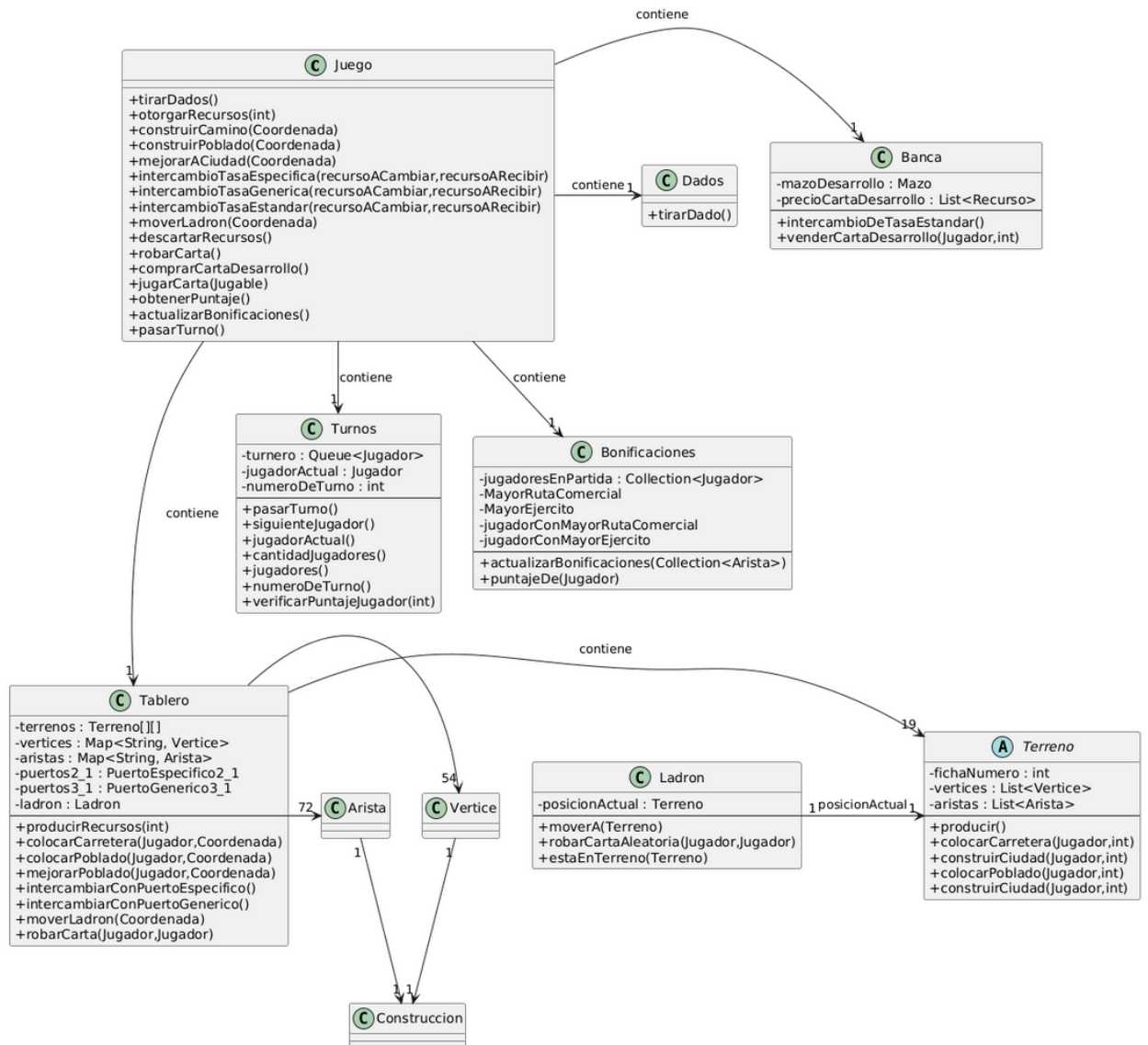


Figura 1: Diagrama clase general

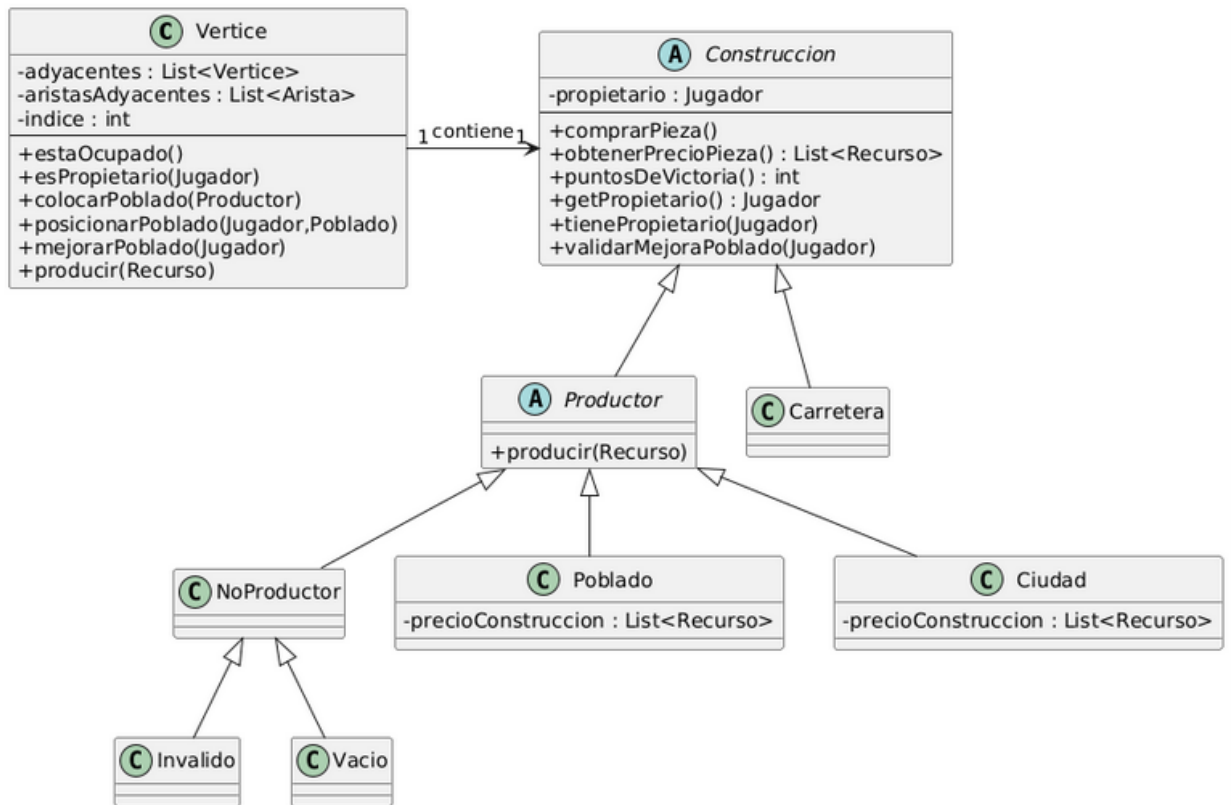


Figura 2: Diagrama relación Vertice-Construccion

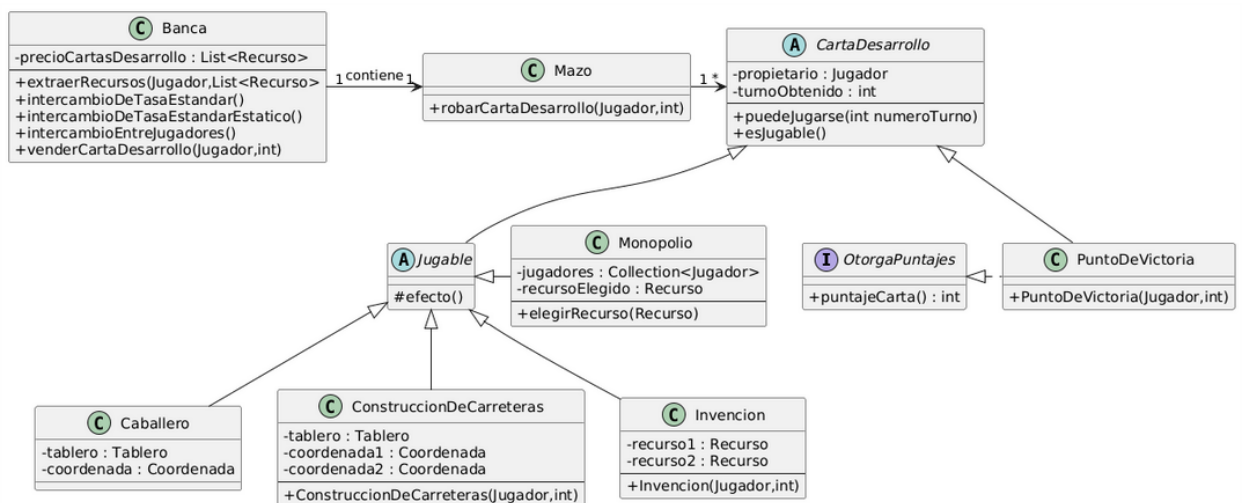


Figura 3: Diagrama Banca, Mazo, CartaDesarrollo

3. Detalles de implementación

3.1. Juego

La clase Juego es la fachada del sistema de Catan. Su responsabilidad es simplificar la interfaz de interacción para el exterior(usuario, jugador). Esta clase no contiene la lógica compleja de las reglas del juego(esa lógica está delegada a Tablero, Jugador, etc.), sino que asegura que la acción correcta se aplique al objeto correcto con el contexto adecuado.

Mantiene referencias a las instancias principales del juego: Turnos, Tablero, Banca y Dados.

Gestiona el ciclo de vida del juego mediante un constructor, inicializa Turnos, Tablero, Banca y Dados.

Traduce acciones de alto nivel(ej: construirPoblado) en llamadas de bajo nivel a los objetos internos, inyectando el contexto necesario(el Jugador actual) que obtiene del turnero. Ejemplo: construirCamino(Coordenada c) se traduce a tablero.colocarCarretera(turnero.jugadorActual(), c).

Gestiona eventos/acciones que afectan a varios jugadores o que tienen un impacto global, como: Tirar Dados y Producción: tirarDados() y la posterior otorgarRecursos(). Movimiento del Ladrón: moverLadron() y la lógica completa de descarte que recorre a todos los jugadores(descartarRecursos), utilizando a turnero.siguienteJugador() para el ciclo.

Provee las interfaces para todas las formas de intercambio:

- Intercambios con el Tablero (puertos): intercambioTasaEspecificas, intercambioTasaGenerica.
- Intercambio con la Banca: intercambioTasaEstandar, comprarCartaDesarrollo.
- Intercambio entre dos jugadores: intercambioEntreJugadores.

Actúa como el único punto de salida para avanzar el juego(pasarTurno()).

Dispara la verificación de la victoria(obtenerPuntaje()).

3.2. Turnos

La clase Turnos actúa como el gestor del flujo temporal del juego. Su responsabilidad es dirigir la secuencia de participación de los jugadores y mantener el estado de avance global de la partida. A través de verificarPuntajeJugador(), sirve como trigger para chequear si el jugador actual ha alcanzado la condición de victoria, delegando el cálculo matemático a la instancia del jugador.

Turnos tiene una cola con todos los jugadores, al jugador actual y el número de turno. pasarTurno() cambia el jugador actual de turno mediante siguienteJugador() y aumenta por 1 el número de turno.

3.3. Tablero

El Tablero es el componente estructural del modelo. Su responsabilidad principal es la inicialización y gestión de la topología del juego, conectando la lógica de matriz(coordenadas x,y) con la lógica de grafos(nodos y aristas compartidos). Tablero tiene una matriz de terrenos, un mapa de vértices y un mapa de aristas. Tiene también las coordenadas de los puertos genéricos y específicos.

Sus responsabilidades específicas son:

Construcción del Mapa(Topología): Instancia la matriz de Terrenos(hexágonos) dándoles forma irregular (3, 4, 5, 4, 3 terrenos por fila). Mapeo de Adyacencias: Es el encargado de determinar que dos terrenos adyacentes compartan las mismas instancias de Vertice y Arista. Esto es crucial para que una carretera o poblado construido entre dos terrenos sea reconocido por ambos. Distribuye aleatoriamente los tipos de terreno(recursos) y las fichas numéricas(probabilidades). Ubica los distintos tipos de puertos en sus vértices correspondientes.

Ciclo de Producción: El método producirRecursos(int numeroFicha) recorre los terrenos y delega en ellos la entrega de recursos a los jugadores, filtrando aquellos bloqueados por el Ladrón.

Fachada de Construcción y Movimiento: Encapsula las acciones de construcción(`colocarPoblado`, `mejorarPoblado`, `colocarCarretera`). Valida si la coordenada existe y delega la validación de reglas específicas al Terreno correspondiente.

Gestión del Ladrón: Controla la instancia única del Ladrón, valida su movimiento(`moverLadron`) y gestiona el bloqueo de producción en el terreno ocupado.

Comercio Marítimo(Puertos): Conoce la ubicación de los puertos(Genéricos 3:1 y Específicos 2:1) y actúa como intermediario para los intercambios comerciales del jugador a través de `intercambiarConPuerto`.

3.4. Jugador

Jugador es la representación del usuario y sus recursos. La clase Jugador modela la entidad que participa activamente en el juego. Su responsabilidad principal es administrar el estado y los activos del usuario(recursos, cartas, construcciones) y ejecutar las acciones que alteran dicho estado.

Entre sus responsabilidades:

Gestión de inventario(recursos): Actúa como un depósito de cartas de de materia prima(`Recurso`). No solo guarda, sino que valida transacciones: métodos como `tieneRecursos(List<Recurso>)` son fundamentales para verificar si el jugador puede costear una construcción antes de intentar realizarla.

Gestión de Construcciones: Mantiene el registro de todas las estructuras(`Construccion`) que posee el jugador en el tablero. Esto es vital para calcular puntajes y, en el caso de `removeConstruccion`, permite la mecánica de mejora(`Poblado` → `Ciudad`).

Manejo de Cartas de Desarrollo: Administra la "mano" de cartas de desarrollo ocultas. Es responsable de la lógica de jugabilidad de la carta: verifica si una carta es válida para ser jugada en el turno actual(evitando jugar cartas recién compradas, salvo puntos de victoria) mediante `tieneCartasJugablesDeTipo`.

Interacción con el Ladrón: Contiene la lógica de penalización por exceso de recursos (`descartarPorLadron`), implementando la regla de perder la mitad de la mano si se tienen más de 7 cartas al salir un 7. Ejecuta la acción de robo(`robarRecursoAleatorio`) sobre otro jugador, manipulando directamente los inventarios de ambos.

Cálculo de Puntos Propios: Cada jugador sabe cuánto vale en términos de puntos de victoria. Suma los puntos de sus construcciones y de sus cartas de Punto de Victoria, compartiendo esta información a la clase `Puntaje()`. Sabe cuánto "vale" el jugador en términos de victoria. Suma los puntos de sus construcciones y de sus cartas de Punto de Victoria, proveyendo esta información a la clase `Puntaje` y/o `Turnos` para determinar el ganador.

3.5. Banca

La clase Banca es la entidad central que representa el suministro ilimitado de recursos y de cartas de desarrollo. Sus responsabilidades se centran en gestionar el comercio básico y la venta de cartas, actuando como contraparte de todos los jugadores.

Intercambio Estándar: Implementa la tasa de intercambio genérica de 4:1. Verifica si el jugador tiene 4 recursos del mismo tipo(`extraerRecursos`) y, de ser así, los elimina para darle 1 recurso diferente. Esto simula el intercambio con el banco en ausencia de puertos.

Validación de Transacciones: El método auxiliar `extraerRecursos` valida si un jugador posee una lista específica de recursos antes de permitir cualquier operación de gasto. Si la validación falla, lanza la excepción `RecursosInsuficientesException`.

Gestión del Mazo de Desarrollo: Contiene y administra el Mazo de cartas de desarrollo (`mazoDeDesarrollo`). Define el costo fijo de las cartas de desarrollo(`precioCartasDesarrollo`: Mineral, Lana, Cereal). El método `venderCartaDesarrollo` coordina la transacción completa: cobra el precio al jugador y le entrega una carta de desarrollo del Mazo.

3.6. Dados

La clase `Dados` es la encargada de simular la tirada de dos dados de seis caras. Su responsabilidad se limita estrictamente a la generación de números aleatorios y a su agregación, sin contener ninguna lógica de juego (como qué sucede si sale un 7).

Su única responsabilidad es la simulación de la tirada de dados: `tirarDado()`: Genera un número entero aleatorio entre 1 y 6 (inclusive). `tirar()`: Es la interfaz principal. Invoca a `tirarDado()` dos veces y devuelve la suma de esos dos resultados, simulando la tirada de los dos dados de Catan (resultado posible entre 2 y 12).

3.7. Terreno

La clase abstracta `Terreno` representa un hexágono del tablero de Catan. Su responsabilidad principal es conectar la topología del tablero con la lógica de producción y construcción. La clase `Terreno` es abstracta ya que modela un concepto genérico (un hexágono de Catan) que nunca existe de forma pura; solo existen sus especializaciones concretas (Bosque, Cerro, Campo, Pastizal, Montaña, Desierto). Esta abstracción asegura que la lógica de juego pueda tratar a todos los hexágonos/terrenos de manera uniforme, mientras que la responsabilidad de especificar el tipo de recurso es delegada a sus clases hijas.

Actúa como un intermediario entre la clase `Tablero` (que sabe la forma del mapa de Catan) y las piezas específicas (Vértices, Aristas, Construcciones).

Almacena las instancias de `Vertice` y `Arista` que delimitan y están contenidas en este hexágono. Es el punto de entrada para que el `Tablero` le diga a qué vértices y aristas debe asignar las referencias compartidas (`agregarVertice`, `agregarArista`).

El método `Terreno.crear()` utiliza el patrón `Factory Method` para seleccionar la clase concreta (Bosque, Cerro, etc.) según el `TerrenoTipo`, y orquestar su instanciación, inyectándole la `fichaNumero` asignada por el `Tablero`.

Sirve como punto de entrada para que el `Tablero` ordene la construcción o mejora de piezas. **Construcción de Piezas:** Recibe la instrucción y el índice local (`indiceVertice` o `indiceArista`) y delega la acción directamente a la instancia de `Vertice` (`colocarPoblado`, `construirCiudad`) o `Arista` (`colocarCarretera`) correspondiente.

La clase es abstracta y usa métodos abstractos (`getTipo()`, `getRecurso()`) que deben ser implementados por las clases hijas (Bosque, Campo, Pastizal, etc.). Esto aplica el patrón `Template Method`, donde el flujo de producción es el mismo para todos (`producir()`), pero el recurso específico que se entrega es determinado por la implementación concreta (por ejemplo, Bosque devuelve `Recurso.MADERA`).

Mediante `tieneNumero(int numeroFicha)`, verifica si su número asociado coincide con el resultado de los dados. Si la ficha coincide, itera sobre todos sus vértices y delega en ellos la acción de entregar el recurso específico del terreno (`this.getRecurso()`) a los jugadores.

3.8. Vertice

En la clase `Vertice` reside la lógica de las reglas de construcción y es el distribuidor final de recursos. El `Vertice` representa un nodo de conexión en el tablero y el contenedor de la `Construccion` del jugador. Su responsabilidad principal es encapsular y gestionar el estado de construcción de un nodo y asegurar que las reglas de distancia y mejora de Poblado a Ciudad sean respetadas.

Atributos y Estructura:

- `construccion` (patrón `Strategy`): Mantiene una referencia a la pieza actual que ocupa el vértice. Puede ser una instancia de `Poblado`, `Ciudad`, `Vacio` o `Invalido`.
- `adyacentes` (Regla de Distancia): Almacena las referencias a los vértices vecinos, útil para hacer cumplir la Regla de Distancia al colocar un nuevo poblado.
- `aristasAdyacentes`: Almacena las referencias a las aristas que se unen en este punto.

Responsabilidades:

- **Regla de la Distancia.** Implementa el método `invalidarAdyacentes()`: Al colocar un poblado, utiliza sus referencias a adyacentes para forzar a los vértices vecinos a cambiar su estado a `Invalido`. De esta forma ningún Jugador puede construir a menos de dos aristas de otro Poblado/Ciudad.
- **Patrón Strategy:** El atributo `construccion` es la estrategia. El `Vertice` delega todo el comportamiento de construcción y producción a la pieza que contiene. `colocarPoblado / mejorarPoblado`: Cambia la estrategia de `Vacio` a `Poblado` y luego a `Ciudad`. La pieza antigua es removida y la nueva es asignada, reflejando la mejora. `esValido()`, `estaOcupado()`, `esPropietario()`: Estos métodos delegan la consulta al objeto `construccion` actual, adaptando el comportamiento del vértice a su estado (ej: un vértice con `Ciudad` tiene un comportamiento diferente a uno con `Vacio`).
- **Producción de Recursos:** El método `producir(Recurso)` es el último eslabón de la cadena de producción. Recibe el recurso del `Terreno` y delega la acción a la `construccion`. Si es una `Ciudad`, la construcción le dirá al Jugador que se agregue dos recursos, uno en el caso de que sea `Poblado`.

3.9. Arista

La clase `Arista` representa la conexión entre dos vértices en el tablero de Catán. En ellas los jugadores pueden colocar sus carreteras. Su responsabilidad principal es gestionar la colocación de Carreteras y verificar la conectividad de una arista con las piezas ya existentes del jugador (regla de la Ruta Continua).

Para cumplir con su rol, la clase `Arista` mantiene las siguientes referencias esenciales:

- `Vértices(vertice1, vertice2)`: Conoce los dos nodos que une. Esto es crucial para consultar si existe un Poblado o Ciudad del jugador en esos extremos.
- `Aristas Adyacentes(adyacentes)`: Conoce las aristas vecinas. Esto es necesario para verificar si ya existe otra Carretera del jugador conectada a esta posición.
- `Propietario(propietario)`: Guarda una referencia al jugador que ha colocado la carretera, si es que está ocupada.

La lógica central de la `Arista` reside en la validación de la conectividad, la cual se resuelve a través de dos condiciones excluyentes:

- **Conexión con Carretera:** La arista debe ser adyacente a otra Carretera del mismo jugador.
- **Conexión con Poblado/Ciudad:** La arista debe estar conectada a un `Vertice` que contenga un Poblado o Ciudad propiedad del mismo jugador.

Si se intenta colocar una carretera en una `Arista` que no cumple ninguna de las dos condiciones, se lanza la excepción `CaminoDesconectadoException`.

3.10. Ladron

La clase `Ladron` representa la pieza que suprime la producción de recursos en un hexágono y facilita el robo de recursos a otros jugadores. Sus responsabilidades son gestionar su posición única en el tablero y mediar en la transferencia aleatoria de un recurso entre dos jugadores.

Se aplicó el patrón `Singleton` para garantizar que solo exista una única instancia de `Ladron` y poder acceder a ella desde otras partes del código que requieran del `Ladron`, sin tener que pasar la instancia de `Ladron` como parámetro.

3.11. Puerto, PuertoEspecifico2_1 y PuertoGenerico3_1

Las clases PuertoEspecifico2_1 y PuertoGenerico3_1 modelan el comercio marítimo con Banca. Estas clases implementan la interfaz común Puerto, aplicando el Patrón Strategy para diferenciar el algoritmo de intercambio basado en la tasa y el tipo de puerto. La clase Tablero actúa como el contexto. El Tablero no contiene directamente el algoritmo, sino que tiene referencias a estas estrategias y delega la llamada de intercambio al puerto específico que el jugador desea usar. La lógica de la ubicación de los puertos es gestionada por la clase Tablero, la cual asocia sus instancias a los Vertices correspondientes.

Su responsabilidad principal es encapsular la lógica y la tasa de intercambio para cada tipo de puerto, y validar que el jugador esté adyacente a dicho puerto para realizar la transacción.

PuertoGenerico3_1 implementa el algoritmo de intercambio 3:1 sin ninguna restricción de tipo de recurso, solo de adyacencia. PuertoEspecifico2_1 implementa el algoritmo de intercambio 2:1, pero con la lógica adicional de validar que el recurso a cambiar coincida con el tipo de puerto adyacente al jugador.

3.12. Puntaje y Bonificaciones

Este grupo de clases encapsula la lógica de la condición de victoria del juego(acumulación de puntos de victoria), separando el cálculo de puntaje base de la determinación de los premios de bonificación.

Bonificaciones gestiona la asignación y el estado de los dos premios globales: Mayor Ruta Comercial y Mayor Ejército. Actúa como el contexto al aplicar el patrón Strategy. Utiliza diferentes estrategias(MayorEjército y MayorRutaComercial) para determinar a los jugadores con las bonificaciones de 2 Puntos de Victoria c/u.

MayorEjército es una estrategia concreta, implementa la lógica para otorgar el premio de 2 PV por el Mayor Ejército.

MayorRutaComercial es otra estrategia concreta, implementa el algoritmo complejo para otorgar el premio de 2 PV por la Mayor Ruta Comercial.

AuxiliarMayorRutaComercial implementa un algoritmo de Búsqueda en Profundidad(DFS) para recorrer las aristas de un jugador y encontrar la ruta continua más larga.

Puntaje se encarga únicamente de la sumatoria final de los Puntos de Victoria de un jugador.

3.13. Cartas de Desarrollo

Este paquete modela la lógica, el comportamiento y las reglas de jugabilidad de las cartas de desarrollo(Caballero, Monopolio, PuntoDeVictoria, Invencion, ConstrucccionDeCarreteras).

Carta Desarrollo(abstracta) define la estructura y estado base común a todas las cartas: el propietario y el turno en que se obtuvieron.

OtorgaPuntajes(interfaz) define la capacidad de contribuir puntos de victoria. Solo la implementa la clase PuntoDeVictoria, pues es la única que aporta puntos de victoria.

Jugable(abstracta) Define el Template Method para jugar cartas. Define el esqueleto inmutable del algoritmo para "jugar una carta".

Las clases concretas heredan de Jugable y aplican el patrón Strategy al implementar el método efecto(). Cada una encapsula una lógica completamente diferente:

- Caballero mueve al ladrón.
- ConstrucccionDeCarreteras coloca dos carreteras sin consumir recursos.
- Invencion otorga dos recursos al Jugador que la juega.
- Monopolio roba todas las unidades de un recurso en específico de las manos de otros jugadores, y se las da al jugador que la juega.

4. Justificación de los patrones de diseño aplicados

4.1. Patrón Singleton

La clase Ladrón es Singleton. En las reglas de Catan, el ladrón es una entidad única. No pueden existir dos ladrones simultáneamente en el tablero. Además, su estado(posición) debe ser accesible globalmente para validar la producción en los terrenos y ejecutar robos.

Evita la instanciación accidental de múltiples ladrones, lo cual rompería la consistencia del estado del juego. Asimismo, proporciona un punto de acceso global, resolviendo el problema de tener que pasar la instancia del ladrón como parámetro a través de múltiples capas de métodos para validar la producción de recursos.

4.2. Patrón Strategy

Vertice(contexto) y Productor/Construcción las estrategias.

Un vértice se comporta de manera distinta si está vacío, si tiene un poblado o si tiene una ciudad. La cantidad de recursos que produce y las reglas de validación cambian dinámicamente dependiendo de la estrategia que tiene vértice.

Elimina la necesidad de estructuras condicionales gigantes dentro de la clase Vertice. Permite que la mejora de una Construcción(de Poblado a Ciudad) sea tan simple como reemplazar el objeto de la estrategia, cumpliendo con el principio Open/Closed. Se pueden agregar distintos tipos de Construcción sin modificar Vertice.

Interfaz Puerto y clases PuertoEspecifico2_1 / PuertoGenerico3_1.

El Tablero(contexto) llama a la estrategia sin saber si la tasa es 2:1 o 3:1; solo sabe que está llamando al método intercambiar() de un Puerto. Si mañana se agrega un puerto 5:1, solo se crea una nueva clase PuertoEspecial5_1 sin tocar el Tablero.

Evita tener que hardcodear las tasas de intercambio en Tablero, y permite agregar nuevos tipos de puertos o modificar las tasas sin alterar el flujo principal de comercio.

Bonificaciones

Bonificaciones(contexto) no tiene que preocuparse por la complejidad del DFS para determinar MayorRutaComercial, ni de criterio utilizado para determinar MayorEjercito. Simplemente le pide a la estrategia que le devuelva el Jugador ganador de esa bonificación, y la estrategia hace el trabajo pesado. Esto permite aislar la lógica compleja en clases específicas.

4.3. Patrón Template Method

La aplican la clase Jugable(CartaDesarrollo y la clase abstracta Terreno).

Se aprecia una estructura fija en varias clases, pero cada una de ellas hace algunas cosas diferentes. En CartaDesarrollo todas las cartas deben validar que no fueron compradas en el turno actual antes de activarse. En Terreno todos los terrenos producen bajo la misma condición(coincidencia de ficha), pero entregan recursos distintos.

Evita la duplicación de código(DRY: Don't Repeat Yourself).

4.4. Patrón Facade

La aplica la clase Juego. Juego provee una interfaz simplificada de todas las acciones que puede tomar un jugador. Le permite al jugador realizar acciones complejas como construir alguna Construcción en una Coordenada sin necesidad de tener que navegar por los vértices y las aristas. Reduce el acoplamiento.

4.5. Patrón Factory Method

Cuando se crean los terrenos al inicializar Tablero. Desacopla la lógica de instanciación de la lógica del Tablero. El Tablero no necesita conocer los constructores específicos de cada clase concreta de Terreno. Simplemente solicita la creación de un tipo y la fábrica se encarga de los detalles de instanciación, facilitando la extensión a nuevos tipos de terreno.

5. Excepciones

CaminoDesconectadoException Se lanza cuando jugador intenta construir una carretera en una posición la cuál no contiene pueblos o carreteras adyacentes del jugador.

CartaNoEsJugableException Se lanza cuando jugador intenta jugar una carta no jugable.

JugadorInvalidoException Se lanza cuando jugador quiere mejorar un poblado que no es suyo.

LadronYaEstaEnTerrenoException Se lanza cuando jugador quiere mover el ladrón al mismo terreno en el que se encuentra actualmente.

NoTieneRecursosParaIntercambioException Se lanza cuando jugador quiere intercambiar con puertos pero no posee suficientes unidades del recurso a entregar.

PosicionInvalidaException Se lanza cuando jugador quiere construir carretera en una arista que ya contiene una carretera de otro jugador.

PuertoNoAdyacenteException Se lanza cuando jugador quiere intercambiar con algún puerto sin tener poblado/ciudad en los puertos.

RecursosInsuficientesException Se lanza cuando jugador quiere construir o quiere comercializar 4:1, pero no tiene los recursos suficientes para hacerlo.

6. Diagramas de secuencia

Se seleccionaron cinco diagramas de secuencia clave que tienen el propósito de mostrar la cadena de comunicación/delegación y la arquitectura interna de nuestro modelo.

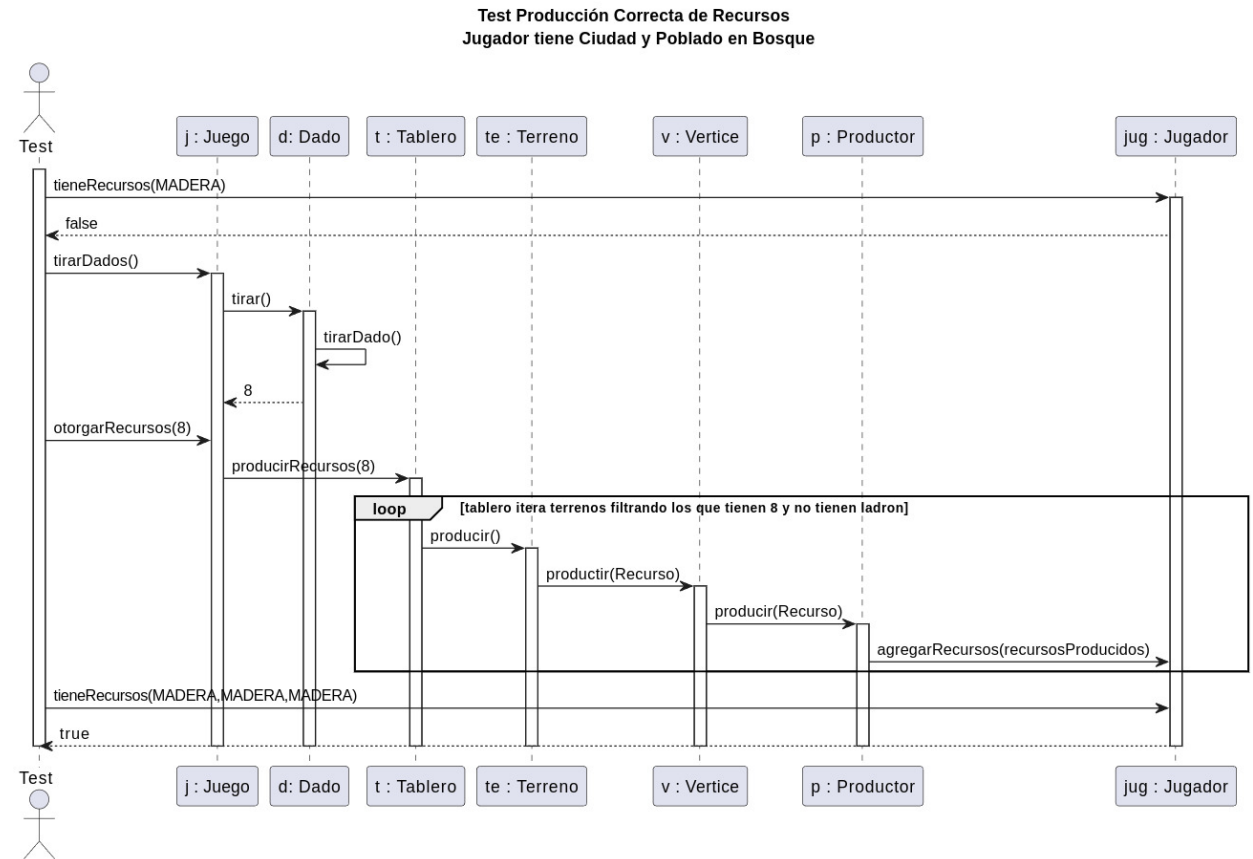


Figura 4: Test 01: Producción de Recursos(Poblado y Ciudad)

Test01 muestra cómo el Tablero coordina la producción y cómo el Vertice aplica el patrón Strategy para diferenciar la cantidad de recursos a entregar(1 o 2) según la Construcción que contenga.

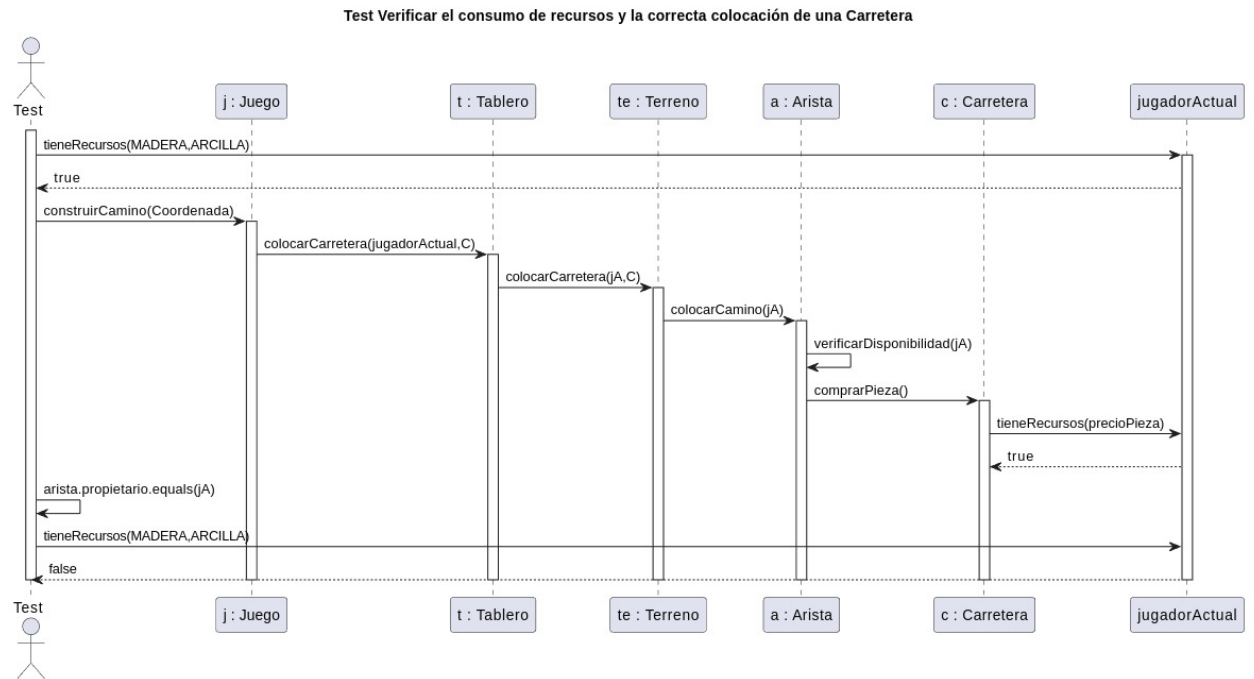


Figura 5: Test 02 Colocación de una Carretera

Test02 detalla la cadena de responsabilidad y las validaciones topológicas: desde el gasto de recursos por el Jugador hasta la comprobación de la Regla de Ruta Continua en la Arista.

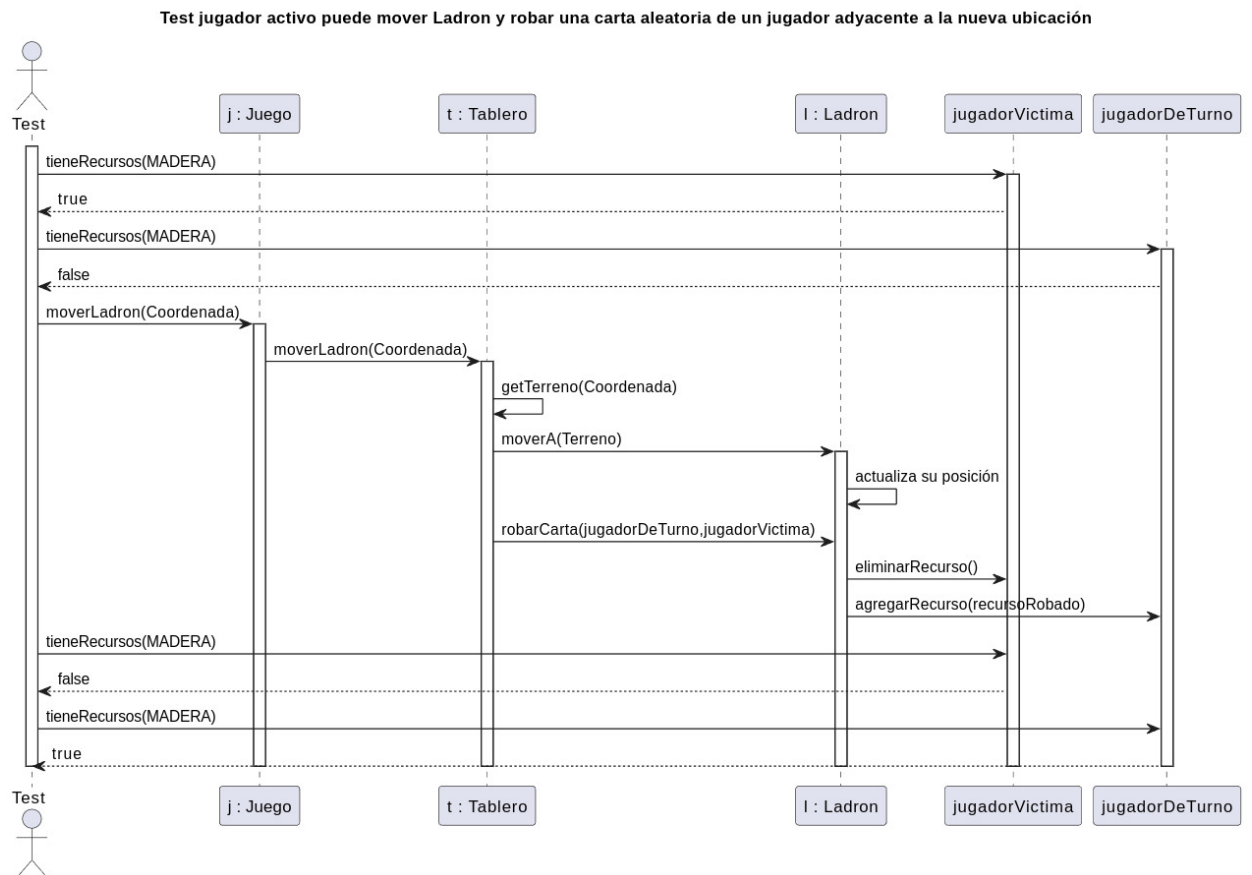


Figura 6: Test 03 Movimiento y Robo del Ladrón

Test03 ilustra el evento global(`moverLadron`) con el patrón Singleton(`Ladron`) y la delegación de la lógica de robo aleatorio al jugador víctima. Verifica que efectivamente el recurso robado intercambie manos.

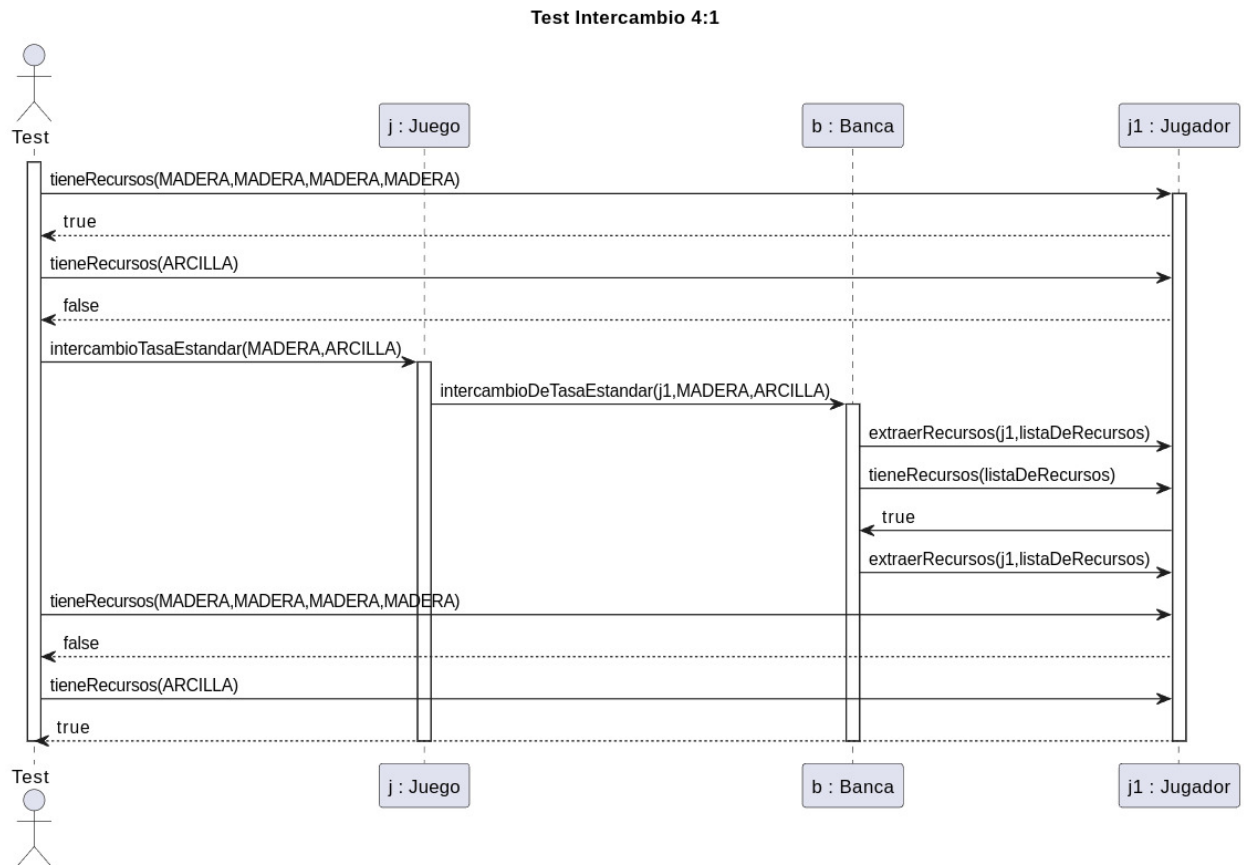


Figura 7: Test 04 Intercambio 4:1 con Banca

Test04 muestra el flujo más simple de intercambio, donde la Banca actúa como la entidad central que ejecuta la tasa de intercambio estándar(4:1) y manipula los recursos del Jugador.

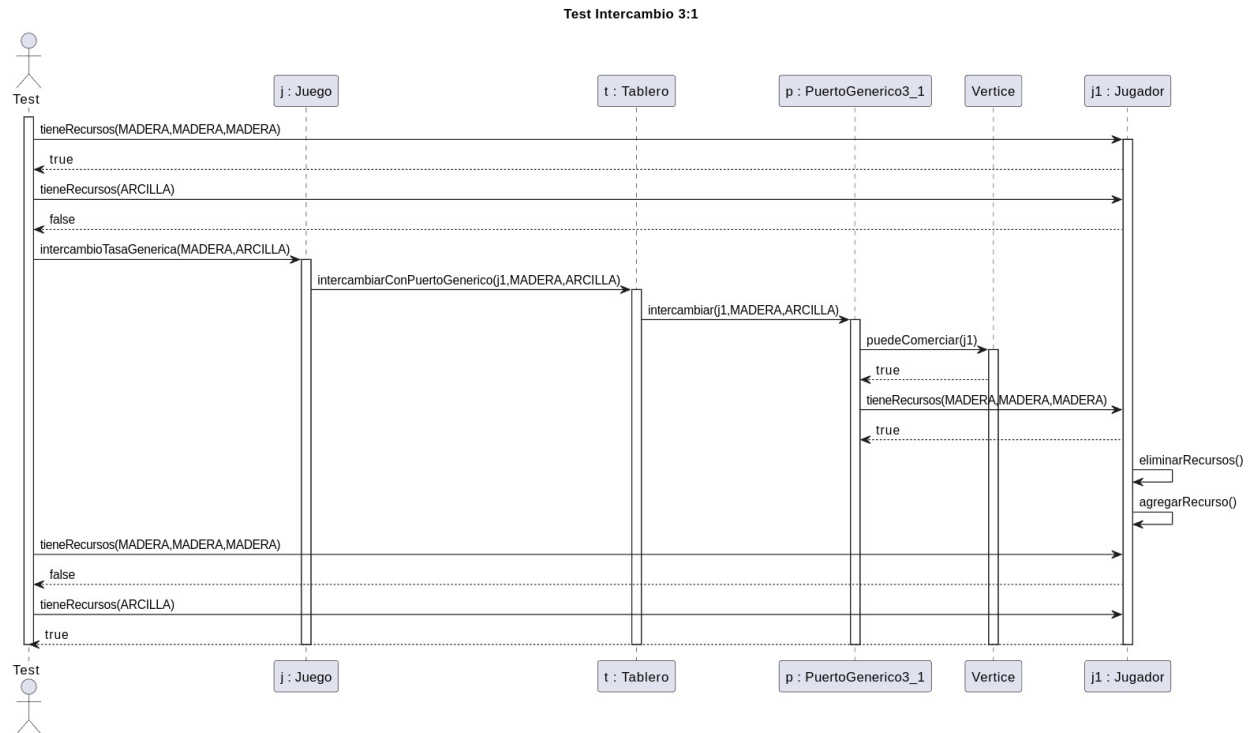


Figura 8: Test 05 Intercambio 3:1 con Puerto Genérico

Test05 el flujo de intercambio 3:1 con PuertoGenerico3_1. Verifica la posibilidad del jugador de acceder a la tasa de intercambio 3:1. Delega la gestión de recursos a quitar y agregar a Jugador.